



UNIVERSIDADE DE FORTALEZA
DEPARTAMENTO DE TECNOLOGIA
CURSO DE TECNÓLOGO EM ANÁLISE E DESENVOLVIMENTO DE SISTEMAS
DISCIPLINA DE DESENVOLVIMENTO DE SOFTWARE EM NUVEM

Relatório Técnico – Catálogo Inteligente

FORTALEZA
2026

GRUPO 10

COMPONENTES:

- Francisca Lindelaine Viana Faustino – 2526839
- Manoel Lucas Gomes Dantas – 2217137
- Pedro Henrique Dutra Galvão – 2425032
- Pedro Yuri Rodrigues Nunes - 2425031
- Vivian Régia Cunha da Silva – 2425202

FORTALEZA
2026

1. Visão Geral do Sistema

O Catálogo Inteligente é uma aplicação web escalável baseada em nuvem, desenvolvida com arquitetura cliente-servidor em três camadas. O sistema permite o gerenciamento completo de um catálogo de produtos com autenticação segura via Firebase Authentication, painel administrativo restrito por custom claims JWT, interface pública de consulta com filtros avançados e upload de imagens para o Firebase Storage.

O caso de uso implementado atende ao modelo de Catálogo Inteligente com Filtros Avançados descrito na proposta, incluindo múltiplos perfis de acesso (usuário comum e administrador), operações CRUD completas e persistência de dados em serviço gerenciado na nuvem.

1.1 Funcionalidades Implementadas

- Autenticação via Firebase Authentication com custom claims (role: admin)
- Proteção de rotas por perfil — usuários comuns acessam o catálogo; admins acessam o painel de gestão
- API RESTful documentada via Swagger/OpenAPI disponível em /docs
- CRUD completo de produtos: criar, listar com paginação, detalhar, editar e excluir
- Upload de imagens para Firebase Storage com validação de tipo (JPG/PNG/WebP) e tamanho (max 5MB)
- Filtros avançados: categoria, faixa de preço, disponibilidade em estoque e busca textual
- Paginação de resultados na listagem pública
- Registro de logs de acesso e erro via middleware dedicado (LoggingMiddleware)
- Validação de dados no backend com Pydantic v2 e CategoryEnum
- Separação de ambientes: development (local) e production (nuvem)

2. Diagrama de Arquitetura em Nuvem

A arquitetura segue o padrão de três camadas com serviços totalmente gerenciados no Google Cloud Platform e Firebase:

Camada	Tecnologia	Serviço de Nuvem
Apresentação	React 18 + TypeScript + Tailwind CSS + Vite	Vercel (catalogo-inteligente-five.vercel.app)
Aplicação	FastAPI + Uvicorn + Python 3.11 + Docker	Google Cloud Run (us-central1)
Imagens Docker	Docker multi-stage (python:3.11-slim)	Google Artifact Registry
Banco de Dados	Cloud Firestore (NoSQL)	Firebase Firestore (gerenciado)
Armazenamento	Firebase Storage (imagens de produtos)	Google Cloud Storage
Autenticação	Firebase Authentication + JWT custom claims	Firebase Auth (gerenciado)
Segredos	credentials.json do service account	Google Secret Manager
CI/CD	GitHub Actions (deploy.yml)	GitHub Actions + Vercel + Cloud Run

2.1 Fluxo de Requisição

- Browser → React SPA (Vercel) → Axios com Bearer token automático → Cloud Run
- Cloud Run → LoggingMiddleware → Dependency de autenticação (verify_token) → Route handler
- Route handler → Firestore Service → Cloud Firestore (persistência dos produtos)
- Upload de imagem → POST /catalog/products/upload-image → Firebase Storage → URL pública retornada
- Credenciais do backend → injetadas via Google Secret Manager no Cloud Run em runtime

3. Tecnologias e Serviços Utilizados

3.1 Frontend

Tecnologia	Versão	Finalidade
React	18.3.1	Framework UI principal
TypeScript	5.4.5	Tipagem estática e segurança de código
React Router DOM	6.23.1	Roteamento SPA com ProtectedRoute
Firebase SDK	10.12.1	Autenticação client-side e custom claims
Axios	1.7.2	HTTP client com interceptors de token
Tailwind CSS	3.4.4	Estilização utilitária responsiva
Vite	5.2.11	Build tool e injeção de variáveis VITE_*
Vitest	1.6.0	Testes de componente frontend

3.2 Backend

Tecnologia	Versão	Finalidade
FastAPI	0.110.0	Framework REST API com OpenAPI automático
Uvicorn	0.29.0	ASGI server de produção
Firebase Admin SDK	6.5.0	Auth, Firestore e Storage no backend
Pydantic	2.6.0	Validação de schemas e CategoryEnum
Docker	python:3.11-slim	Containerização multi-stage
Pytest	8.2.0	Testes automatizados de API

4. Estratégia de Deploy e CI/CD

4.1 Containerização — Docker Multi-Stage

- Stage 1 (builder): instala dependências Python sobre python:3.11-slim com gcc
- Stage 2 (runtime): copia apenas os pacotes compilados e o código da aplicação
- Resultado: imagem enxuta sem ferramentas de build, porta 8080, comando uvicorn
- Imagem publicada no Google Artifact Registry: us-central1-docker.pkg.dev/catalogo-unifor-1629/catalogo/backend

4.2 Pipeline CI/CD — GitHub Actions (deploy.yml)

O pipeline possui 3 jobs executados sequencialmente a cada push na branch main:

Job	Duração	Ações Executadas	Dependência
Testes Backend	~19s	pip install + pytest 5 testes	Nenhuma
Build e Deploy Backend	~1m 36s	docker build + push Artifact Registry + gcloud run deploy	Testes passarem
Deploy Frontend	~2m 18s	npm install + vite build + vercel --prod	Testes passarem

4.3 Separação de Ambientes

- development: variáveis locais via arquivo `_env`, backend em localhost:8000, frontend em localhost:5173
- production: variáveis injetadas via GitHub Secrets no CI e Secret Manager no Cloud Run
- Credenciais nunca commitadas — `.gitignore` inclui `_env`, `credentials.json` e `.env`
- CORS configurado por variável `ALLOWED_ORIGINS` para restringir origens por ambiente
- Firebase inicializado antes dos imports dos routers para garantir disponibilidade em runtime

5. Papéis e Contribuições da Equipe

Integrante	Papel	Principais Contribuições
[Nome 1]	Arquiteto(a) de Software em Nuvem	Definição da arquitetura GCP/Firebase, Secret Manager, Cloud Run
[Nome 2]	Desenvolvedor(a) Back-end	FastAPI, Firestore service, autenticação, upload de imagens
[Nome 3]	Desenvolvedor(a) Front-end	React, AuthContext, painel admin, componentes UI, vite-env.d.ts
[Nome 4]	Engenheiro(a) DevOps	GitHub Actions, Artifact Registry, Cloud Run, Vercel deploy
[Nome 5]	Qualidade e Testes	Pytest, conftest.py mock Firebase, vitest, Swagger

Preencha os nomes reais dos integrantes antes de entregar.

6. Dificuldades Encontradas e Soluções Adotadas

#	Dificuldade	Solução Adotada
1	Token Firebase retornando 401 — token estático no Swagger não era JWT válido	Geração de ID Token real via Firebase REST API (signInWithPassword) e configuração da FIREBASE_WEB_API_KEY
2	403 Forbidden — usuário sem custom claim admin	Firebase Admin SDK com set_custom_user_claims() e forceRefresh no AuthContext do frontend
3	Variáveis de ambiente sem prefixo VITE_ no Vercel causando Firebase invalid-api-key	Recriação de todas as vars com prefixo VITE_ e arquivo vite-env.d.ts para tipagem TypeScript
4	Firebase inicializado após imports dos routers causando 500 no upload	Reorganização do main.py para inicializar firebase_admin antes dos imports dos routers
5	Bucket Firebase Storage com nome incorreto (404 no upload)	Ativação do Firebase Storage no Console e uso do nome correto .firebasestorage.app
6	GCR não permitia push sem repositório pré-criado	Migração para Google Artifact Registry com repositório criado via gcloud artifacts repositories create
7	Compute service account do Cloud Run sem acesso ao Secret Manager	Concessão de roles/secretmanager.secretAccessor para 464009045772-compute@developer.gserviceaccount.com
8	Pytest falhando no CI por Firebase inicializar na importação do módulo	Criação de confest.py que mocka sys.modules do firebase_admin antes de qualquer import da app
9	page_size=100 retornando 422 — limite máximo era 50 no Query do FastAPI	Ajuste do parâmetro le=200 no endpoint de listagem de produtos

Frontend: catalogo-inteligente-five.vercel.app

Backend: catalogo-backend-cphx32ua4q-uc.a.run.app

Unifor — Fortaleza, CE — 2026